

Portfolio Optimization with TD3 Algorithm Using Technical Indicators

No Author Given

No Institute Given

Abstract. We explore the application of the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm to portfolio optimization. Financial markets present high-dimensional and non-stationary environments where traditional reinforcement learning methods often suffer from value overestimation and instability. Unlike other approaches that use OHLCV (Open, High, Low, Close, Volume) data, we evaluate our approach using only a diverse set of normalized technical indicators. Such indicators are derived from OHLCV data; however, they are widely used by investors who prefer technical analysis to fundamental analysis. Our experimental results demonstrate that the agent achieves stable convergence. We further analyze the policy’s behavior using the Herfindahl–Hirschman Index (HHI) and portfolio turnover, revealing a shift toward more concentrated capital allocation during the inference phase.

Keywords: Algorithmic Trading · Deep Reinforcement Learning · Portfolio Optimization · TD3

1 Introduction

Algorithmic trading and portfolio optimization have undergone significant transformation with the advent of Deep Reinforcement Learning (DRL) [27]. Traditional portfolio management often relies on mean-variance optimization, which typically requires accurate estimation of expected returns and covariance matrices, tasks that are notoriously difficult in volatile markets. DRL offers a paradigm shift by enabling agents to learn optimal trading policies directly from high-dimensional market data through interaction and feedback.

However, applying DRL to finance is not without challenges. Actor-critic methods, such as Deep Deterministic Policy Gradient (DDPG) [14], are prone to overestimation bias and high variance, which can lead to suboptimal policies and unstable training. To address these issues, we employ the TD3 algorithm [12], which introduces clipped double Q-learning, delayed policy updates, and target policy smoothing. This research focuses on constructing a robust DRL framework for portfolio selection that specifically accounts for transaction costs and investor risk aversion. Unlike other approaches that use OHLCV (Open, High, Low, Close, Volume) data, we evaluate our approach using a diverse set of normalized technical indicators. Such indicators are derived from OHLCV data; however, they are widely used by investors who prefer technical analysis to fundamental analysis.

2 Related Work

Value-based reinforcement learning methods first estimate a value function and then derive a policy from it. Thus, the quality of the policy depends entirely on the accuracy of the estimated action-value function $Q(s, a)$. In high dimensional state or action spaces, tabular representations of Q -values become infeasible, so the function approximation with neural networks is used. This introduces approximation error stemming from network architecture and training dynamics. Such error appears as noise in the Q -value estimates. When actions are selected via a maximization step, $\max_a Q(s, a)$, this noise induces a positive bias, causing systematic overestimation. Thrun and Schwartz [28] first identified this effect, and Hasselt [11] later formalized it in the context of Double Q -learning (Double QL). As a result, Q -value overestimation in value-based methods can lead to suboptimal policies and unstable or even divergent training, especially in high-dimensional environments.

Prior work has examined bias-variance challenges in reinforcement learning. Pendrith [20] showed that RL algorithms lack adaptive mechanisms to control estimator variance, leading to instability in noisy or non-Markov environments. They introduced *ccBeta*, an adaptive per-parameter learning-rate method based on online error autocorrelation, validated through theory, simulation, and real-robot experiments.

Hasselt [11] analyzed how uncertainty in estimated Markov decision process (MDP) transitions and rewards propagates through the Bellman equation. Using second-order approximations, they derived closed-form bias and variance expressions for value estimates and showed that variance typically dominates, enabling practical confidence intervals and statistically sound policy evaluation.

Fujimoto et al. [9] studied function-approximation error in actor-critic methods, proposing an improved Deep Deterministic Policy Gradient (DDPG) [14]. While Double DQN [29] and Double Q -learning [11] mitigate overestimation bias, Double Q -learning can introduce high variance that still harms policy performance. To address this, they proposed Clipped Double Q -learning, which uses the minimum of two independently learned Q -value estimates to prevent optimistic errors from propagating.

Additional work explored variance-reduction strategies in Q -learning, including averaging past Q -networks [3] and soft-update mechanisms to avoid overfitting noisy early estimates [8]. Temporal-difference methods also suffer from compounding errors due to bootstrapping on imperfect estimates, with mitigation typically achieved through multi-step returns or hybrid Monte-Carlo/TD updates.

Portfolio construction can be optimized using Machine Learning (ML), RL, and Deep RL (DRL). ML supports forecasting, asset pre-selection, and decision-making. Rubesam [24] applies ML-based portfolio construction and compares it with traditional methods; Mavruk [15] uses ML to detect investor herding; and Fereydooni and Mahootchi [7] employ ML for algorithmic trading in currency markets.

RL is well-suited to dynamic financial markets due to its sequential decision-making framework. RL methods are grounded in the MDP [1, 19], with algorithms such as QL, Deep QL (DQL), DDPG, and PPO representing common extensions. DDPG is particularly important for continuous-action portfolio optimization [14]. High-dimensional portfolio DQL models have been explored by Pigorsch and Schäfer [22], while multi-agent DQL approaches were developed by Park et al. [18] and Shavandia and Khedmati [25].

DRL has become a strong approach for portfolio optimization, combining deep learning’s representation power with RL’s sequential decision-making. This enables strategies that adapt to dynamic, stochastic, and high-dimensional markets. Bühler et al. [5] introduced Deep Hedging, a DRL method incorporating risk constraints and transaction costs, while Li [13] proposed a DRL framework for online portfolio selection that explicitly models transaction costs. Yang [31] proposed an ensemble DRL method for automated trading suited to high-dimensional markets. Zhao [33] introduced an asset-correlation-based DRL approach that improves portfolio selection by leveraging cross-asset relationships. Cui [6] incorporated CVaR-based risk and investor risk aversion into a DRL portfolio-construction framework. Betancourt and Chen [4] developed a DRL model capable of handling markets with a dynamic number of tradable assets, a key requirement for transaction-cost-aware trading.

Earlier DRL portfolio methods often ignored transaction costs or lacked explicit risk-aversion modeling. These limitations were addressed by integrating mean-variance constraints into DRL [10, 30], developing cost-sensitive DRL portfolio models [32], and incorporating asset variability and correlations through repeated DRL decision processes [16].

3 Methodology

This section outlines the core principles of the *TD3 algorithm* applied in *Deep reinforcement learning for portfolio selection* [12].

In *value-based* RL, function approximation errors often lead to overestimated value functions and suboptimal policies. These issues persist in an *actor-critic* setting. Fujimoto et al. [9] address this problem with TD3, which builds on the Deterministic Policy Gradient (DPG) algorithm [26], an actor-critic approach that trains a deterministic policy using learned value estimates. When a policy is updated via gradient ascent on an approximate value function, small update steps locally maximize the estimated value, but not necessarily the true value. If the critic is systematically optimistic, the policy update will favor actions with positive estimation error, causing the policy’s true performance to be overestimated. This explains why overestimation bias can arise in actor-critic methods even without explicit action maximization. To reduce overestimation bias in actor-critic methods, the TD3 authors introduced a clipped variant of Double QL. In Double QL, action selection (the greedy step) and action evaluation (target computation) are separated across two value estimators to prevent shared error accumulation. The Double DQN idea of separating selection (online net-

work) from evaluation (target network) is specific to value-based DQN, while the clipped variant adapts this separation to actor-critic settings.

Target update using the clipped variant of Double QL is defined as:

$$y_1 = r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \pi_{\phi_1}(s')), \quad (1)$$

where y_1 denotes the target value used to update critic $Q_{\theta'}$, r is the reward observed after taking action a from state s , γ with $0 \leq \gamma < 1$, is the discount factor, $\min_{i=1,2}(\cdot)$ is the clipped Double Q operator, applied over the two target critics and $Q_{\theta'_i}(\cdot)$ are the target critic networks parametrized by π_{ϕ_1} and ϕ'_i is the actor network (policy) used when constructing the target for critic 1.

Another key issue is variance. High-variance value estimates produce noisy policy-gradient updates, destabilizing learning; variance not only contributes to overestimation but also harms overall training stability. TD learning amplifies this problem because each update bootstraps on an approximate next-state value, leaving a residual TD error that accumulates over time. To improve stability, target networks are commonly used. Since deep function approximators require many gradient steps to converge, slowly updated target networks provide a more stable learning objective and make better use of experience data. The weights of the target network are slowly updated by τ at each time step using $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$, where τ is the soft-update rate that determines how quickly the target network's weights move toward the online network's weights.

A limitation of deterministic policies is their tendency to overfit to sharp peaks in the value function. Because the critic's targets depend on a single deterministic action, they become highly sensitive to function-approximation errors, increasing variance. To mitigate this, the authors introduce target policy smoothing, a regularization strategy inspired by SARSA [27]. Instead of using one deterministic action to compute the critic's target, small clipped noise is added to the target action, effectively averaging value estimates over nearby actions. This enforces the idea that similar actions should yield similar values, producing smoother value functions and more stable policy updates.

The target update with target policy smoothing is defined as follows:

$$y = r + \gamma Q_{\theta'}(s', \pi_{\phi'}(s') + \varepsilon), \quad \varepsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c), \quad (2)$$

where y denotes the smoothed target used to update the critic; r is the reward obtained after taking action a from state s ; γ , with $0 \leq \gamma < 1$, is the discount factor; $Q_{\theta'}(\cdot)$ is the target critic network parameterized by θ' ; $\pi_{\phi'}(s')$ is the target actor generating the next-state action, parameterized by ϕ' ; ε is the target policy smoothing noise drawn from a Gaussian distribution $\mathcal{N}(0, \tilde{\sigma})$ and clipped element wise to the interval $[-c, c]$; c controls the maximum magnitude of the smoothing perturbation; and $\tilde{\sigma}$ denotes the standard deviation of the smoothing noise prior to clipping.

The TD3 algorithm is depicted in Fig. 1, where $\tilde{a}$ denotes the noisy target action used for target policy smoothing; ε denotes the target-smoothing noise; $\tilde{\sigma}$ is the standard deviation of this noise; d is the policy-delay parameter; τ is the target-update rate; and N denotes the mini-batch size.

Algorithm 1 TD3

```

Initialize critic networks  $Q_{\theta_1}, Q_{\theta_2}$ , and actor network  $\pi_\phi$ 
with random parameters  $\theta_1, \theta_2, \phi$ 
Initialize target networks  $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$ 
Initialize replay buffer  $\mathcal{B}$ 
for  $t = 1$  to  $T$  do
    Select action with exploration noise  $a \sim \pi_\phi(s) + \epsilon$ ,
     $\epsilon \sim \mathcal{N}(0, \sigma)$  and observe reward  $r$  and new state  $s'$ 
    Store transition tuple  $(s, a, r, s')$  in  $\mathcal{B}$ 

    Sample mini-batch of  $N$  transitions  $(s, a, r, s')$  from  $\mathcal{B}$ 
     $\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \bar{\sigma}), -c, c)$ 
     $y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \tilde{a})$ 
    Update critics  $\theta_i \leftarrow \text{argmin}_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$ 
    if  $t \bmod d$  then
        Update  $\phi$  by the deterministic policy gradient:
         $\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$ 
        Update target networks:
         $\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$ 
         $\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$ 
    end if
end for

```

Fig. 1: Twin Delayed Deep Deterministic policy gradient algorithm (TD3) [9].

In our work we apply a model-free DRL framework introduced by Jiang [12]. DRL leverages neural networks as function approximators, enabling the automatic extraction of complex patterns and interactions across diverse assets. The portfolio value at time t is given by:

$$P_t = P_{t-1} + p_t^\top k_t, \quad (3)$$

where P_{t-1} denotes the portfolio value from the previous period, $p_t = [p_{1,t}, \dots, p_{N,t}]$ represents the vector of closing prices of all N assets at time t , and $k_t = [k_{1,t}, \dots, k_{N,t}]$ is the vector of trading actions at time t , where each $k_{n,t}$ specifying whether asset n is bought, held, or sold.

In the portfolio selection problem, transaction costs must be incorporated into every trading operation. Following [31], the transaction cost at time t is defined as:

$$c_t^{\text{tran}} = \xi \times p_t^\top k_t, \quad (4)$$

where ξ is a positive constant representing the transaction cost rate associated with executing a new trade. The term $p_t^\top$ denotes the transpose of the asset price vector p_t , and $k_t = [k_{1,t}, \dots, k_{N,t}]$ is the portfolio decision vector - also referred to as the trading action.

Beyond transaction costs, the objective function also incorporates underlying portfolio risk, captured by the portfolio variance σ_t^2 , together with the investor's

risk aversion coefficient β . The risk penalty is introduced through the cost term:

$$c_t^{\text{risk}} = \beta \sigma_t^2. \quad (5)$$

The portfolio trading decision is formulated as a MDP, represented by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r)$, where $\mathcal{S}$ denotes the state space, defined by a vector of state variables $s_t = [p_t, m_{t-1}]$, $\mathcal{A}$ is the action space containing all feasible actions $a_t \in \{\text{buy}, \text{hold}, \text{sell}\}$, $\mathcal{P}$ denotes the state transition probability, r the reward function and m_t is the vector of asset holdings (number of shares) at time t .

The trading strategy is characterized by a portfolio policy $\pi(a_t, s_t)$, and the reward function $r_t(s_t, a_t, s_{t+1})$ is obtained by applying this policy once the next state s_{t+1} is realized. The objective reward function incorporates both risk and transaction costs:

$$r_t = p_t^\top m_t - \beta \sigma_t^2 - \xi p_t^\top |k_t|, \quad (6)$$

where $p_t^\top m_t$ represents the total market value of the portfolio. The term $\beta \sigma_t^2$ denotes the volatility penalty reflecting the investor's risk aversion, while $\xi p_t^\top |k_t|$ accounts for the transaction costs associated with executing trades.

The agent aims to learn an optimal trading policy $\pi(s_t, a_t)$ that selects actions maximizing the reward. Policy performance is evaluated through the action-value function $Q^\pi(s_t, a_t)$, or Q-function, which can be expressed via the Bellman equation as described by Wiering and Otterlo [17]:

$$Q^\pi(s_t, a_t) = E_{s_{t+1}} [r_t(s_t, a_t, s_{t+1}) + \gamma E_{a_{t+1}} [Q^\pi(s_{t+1}, a_{t+1})]]. \quad (7)$$

Q-learning is a widely used RL method that updates the action-value function $Q^\pi(s_t, a_t)$ recursively to maximize cumulative rewards and derive the optimal trading policy. The temporal-difference (TD) error quantifies the gap between predicted and actual Q-values as the agent transitions between states, and the Q-value is updated by scaling this error with the learning rate [17]:

$$\begin{aligned} Q_\pi^{\text{New}}(s_t, a_t) &\leftarrow Q_\pi^{\text{Old}}(s_t, a_t) \\ &+ \alpha \left(r_t(s_t, a_t, s_{t+1}) + \gamma \max_{a_{t+1} \in \mathcal{A}} Q_\pi(s_{t+1}, a_{t+1}) - Q_\pi^{\text{Old}}(s_t, a_t) \right). \end{aligned} \quad (8)$$

4 Experimental Setup

We utilize historical data sourced from the Alpaca API Platform's Market Data API [2]. Our analysis focuses on the period 2016 to 2023, using 30-minute interval data. The data from 2016 to 2019 are used for training and the data from 2019 to 2023 for testing.

Our experiments were conducted on the following ten tickers: AAPL, MSFT, AMZN, GOOGL, META, TSLA, NVDA, JPM, JNJ, XOM. We are taking into

consideration only long positions, acknowledging that the exclusion of short positions may reduce the potential return compared to strategies that utilize both long and short positions.

4.1 Indicators

Unlike Jiang [12], which uses OHLC data as input, our model uses normalized technical indicators. Our model is trained on 17 indicators to enhance decision-making and improve performance. The indicators include RSI and Rate of change (ROC) with a 20-day window; MACD with parameters (12, 26, 9); Simple Moving Average (SMA) and Exponential Moving Average (EMA) with periods of 20, 50, 100, and 200; Bollinger Bands using a 20-day window and std. dev = 2.0; Keltner Channels with a 20-day window, 10-day Average True Range (ATR) window, and multiplier of 2.0; and TTM Squeeze with a 20-day window, 20-day ATR window, and multiplier of 1.5. Additional indicators are TSI (slow 25, fast 13), MFI, VWAP, ATR, ADX, and CMF, each with a 14-day window, along with AROON using a 25-day window. We also take into consideration the closing price.

4.2 Training and Testing Setup

The differences between the training and testing modes lie primarily in the settings that influence the agent’s behavior. The core processing logic remains the same in both cases, with the differences summarized in Table 1.

Table 1: Differences Between Training and Testing Modes.

Property	Training Mode	Testing Mode
Exploration (noise)	Exploration noise is used when selecting actions.	Noise is not used so it does not influence action selection.
Agent learning	The TD3 parameters are updated.	Learning is disabled; parameters remain unchanged.
Replay buffer	Transitions are stored and sampled for mini-batch learning.	Replay buffer is not used for learning, so storing samples is unnecessary.
Agent initialization	The agent is initialized without prior experience.	The trained model is loaded.

The number of training episodes is set to 50, the learning rate is 0.0003 and the batch size is 128. Both the actor and critics use the Adam optimizer, and critics trained using Mean Squared Error (MSE) loss. ReLU is applied as an activation function in hidden layers and the Sigmoid activation function is used in the output layer.

In order to provide a robust performance estimate, we use 5 independent training and testing runs (herein we refer to them as run0–run4).

4.3 Implementation Details

Our implementation uses Python 3.12 and PyTorch as a machine learning framework. Pandas and NumPy handle data processing and numerical computations. The experiments are run on two devices: MacBook Pro (M3 Pro, 36 GB RAM), and a PC with Ubuntu 22 LTS OS and GPU (CUDA): NVIDIA GeForce RTX 3090.

Both TD3 actor and critic network is a three-layer MLP that flattens the input, processes it through two ReLU-activated hidden layers, and outputs continuous actions in $[0,1]$ via a Sigmoid layer. The actor network uses entmax [21] to normalize portfolio output weights. The algorithm uses an initial noise set to 0.3, the final noise is 0.05, and the noise annealing period is 500, defining how long it takes to gradually reduce the exploration noise from its initial value to its final value. The replay buffer size is set to 100,000 and the initial cash is 10,000.

5 Experimental Results and Performance Analysis

The key challenge in designing portfolio optimization approaches using RL is their overall convergence. The results related to convergence are presented in Fig. 2. This figure shows how the cumulative reward evolved over 500 episodes, the upward trend indicates that the agent is successful in learning portfolio weights to achieve higher returns.

Fig. 3 displays the weight distribution in the final training episode. Instead of allocating capital evenly, the agent adjusts weights based on market conditions, aiming to maximize the reward, while responding dynamically to market changes. We can see that there are time steps when the portfolio is more weighted towards cash and other time steps when the portfolio is weighted towards stock allocation.

As shown in Fig. 3, the training phase exhibits some stochastic noise. The exploration–exploitation trade-off effects are reflected in frequent portfolio rebalancing. In contrast, Fig. 4 reveals a more stable policy. The “vertical striping” observed during training is replaced by coherent, contiguous blocks of capital allocation. This shift indicates that the agent has transitioned from a stochastic exploratory phase to a more deterministic exploitation of learned market patterns.

To quantitatively assess the behavior of the TD3 agent, we apply a formal statistical comparison between the training phase and the deterministic test phase. We focus on two critical metrics: the Herfindahl-Hirschman Index (HHI) to measure asset concentration, and the Portfolio Turnover to measure temporal policy stability.

The concentration of weights is measured using the HHI, defined as:

$$HHI_t = \sum_{i=1}^N (w_{i,t})^2, \quad (9)$$

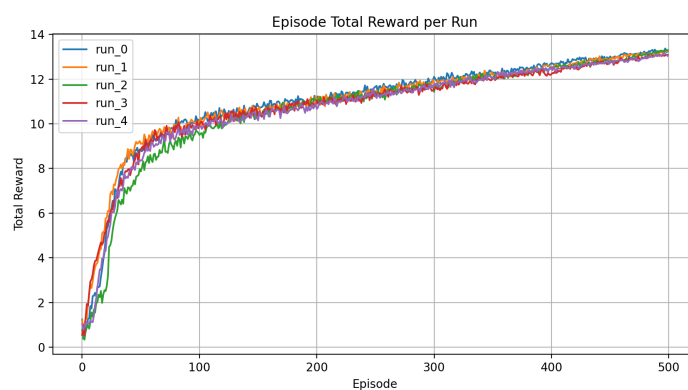


Fig. 2: The cumulative reward during training.

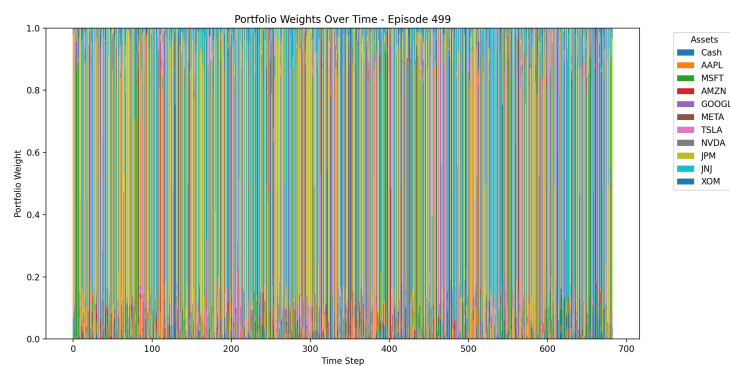


Fig. 3: Portfolio weights during training (run_0).



Fig. 4: Portfolio weights during testing (run_0).

where $w_{i,t}$ represents the weight of asset i at time t .

The Portfolio Turnover, which quantifies the rebalancing frequency and the magnitude of change in the agent’s actions, is defined as:

$$\mathcal{T} = \frac{1}{T-1} \sum_{t=1}^{T-1} \sum_{i=1}^N |w_{i,t+1} - w_{i,t}| \quad (10)$$

where T is the total number of time steps, N is the number of assets, and $w_{i,t}$ represents the weight of asset i at time t .

The test episode exhibited a mean HHI of 0.764 ($\sigma = 0.162$), representing a 15.58% increase over the training mean. This suggests that during inference, the agent converges on a "winner-take-all" strategy, significantly reducing portfolio diversification.

The results indicate a substantial degradation in stability during the test phase. The mean turnover rose from 1.040 during training to 1.381 in Fig. 3 (a 32.79% increase). This high turnover confirms that the deterministic policy is highly reactive to state-space noise, leading to excessive trading activity.

6 Conclusion

We have presented an application of the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm for the problem of continuous portfolio optimization. By utilizing normalized technical indicators as state inputs and incorporating transaction costs and risk aversion into the reward function, we developed an agent capable of navigating high-dimensional asset spaces.

Our experimental results demonstrate that the TD3 agent successfully achieves convergence, as evidenced by the upward trend in cumulative rewards during the training phase. A critical observation from our analysis is the qualitative shift in policy behavior between training and testing. While training exhibits significant stochasticity due to exploration noise, the inference phase reveals a highly concentrated "winner-take-all" strategy, confirmed by a 15.58% increase in the Herfindahl-Hirschman Index (HHI). Furthermore, we observed that the deterministic policy maintains a high turnover rate (an increase of 32.79%), indicating a high sensitivity to state-space noise when exploration noise is removed.

Our future research will address how portfolio concentration can be mitigated. We also intend to validate our results with QuantConnect [23], an event-based simulator for validation and testing of trading strategies.

References

1. Almahdi, S., Yang, S.Y.: An adaptive portfolio trading system: A risk-return portfolio optimization using recurrent reinforcement learning with expected maximum drawdown. *Expert Systems with Applications* **87**, 267–279 (2017)
2. Alpaca Markets: Alpaca market data api documentation. <https://docs.alpaca.markets/docs/about-market-data-api> (2024), accessed: 2026-02-12

3. Anschel, O., Baram, N., Shimkin, N.: Averaged-dqn: Variance reduction and stabilization for deep reinforcement learning. In: International conference on machine learning. pp. 176–185. PMLR (2017)
4. Betancourt, C., Chen, W.H.: Deep reinforcement learning for portfolio management of markets with a dynamic number of assets. *Expert Systems with Applications* **164**, 114002 (2021)
5. Buehler, H., Gonon, L., Teichmann, J., Wood, B.: Deep hedging. *Quantitative Finance* **19**(8), 1271–1291 (2019)
6. Cui, T., Ding, S., Jin, H., Zhang, Y.: Portfolio constructions in cryptocurrency market: A cvar-based deep reinforcement learning approach. *Economic Modelling* **119**, 106078 (2023)
7. Fereydooni, A., Mahootchi, M.: An algorithmic trading system based on a stacked generalization model and hidden markov model in the foreign exchange market. *Global Finance Journal* **56**, 100825 (2023)
8. Fox, R., Pakman, A., Tishby, N.: Taming the noise in reinforcement learning via soft updates. arXiv preprint arXiv:1512.08562 (2015)
9. Fujimoto, S., Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods. In: International conference on machine learning. pp. 1587–1596. PMLR (2018)
10. García-Galicia, M., Carsteanu, A.A., Clempner, J.B.: Continuous-time reinforcement learning approach for portfolio management with time penalization. *Expert Systems with Applications* **129**, 27–36 (2019)
11. Hasselt, H.: Double q-learning. *Advances in neural information processing systems* **23** (2010)
12. Jiang, Y., Olmo, J., Atwi, M.: Deep reinforcement learning for portfolio selection. *Global Finance Journal* **62**, 101016 (2024)
13. Li, B., Wang, J., Huang, D., Hoi, S.C.: Transaction cost optimization for online portfolio selection. *Quantitative Finance* **18**(8), 1411–1424 (2018)
14. Lillicrap, T.P.: Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971 (2015)
15. Mavruk, T.: Analysis of herding behavior in individual investor portfolios using machine learning algorithms. *Research in International Business and Finance* **62**, 101740 (2022)
16. Olschewski, S., Diao, L., Rieskamp, J.: Reinforcement learning about asset variability and correlation in repeated portfolio decisions. *Journal of Behavioral and Experimental Finance* **32**, 100559 (2021)
17. Otterlo, M.v., Wiering, M.: Reinforcement learning: State-of-the-art. *Adaptation, Learning, and Optimization* **12** (2012)
18. Park, H., Sim, M.K., Choi, D.G.: An intelligent financial portfolio trading strategy using deep q-learning. *Expert Systems with Applications* **158**, 113573 (2020)
19. Peck, J., Yang, H.: Investment cycles, strategic delay, and self-reversing cascades. *International economic review* **52**(1), 259–280 (2011)
20. Pendrith, M.D., Ryan, M.R., Sammut, C.: Estimator variance in reinforcement learning: Theoretical problems and practical solutions. University of New South Wales, School of Computer Science and Engineering (1997)
21. Peters, B., Niculae, V., Martins, A.F.T.: Sparse sequence-to-sequence models. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. pp. 1504–1519. Association for Computational Linguistics, Florence, Italy (2019), <https://aclanthology.org/P19-1146>

22. Pigorsch, U., Schäfer, S.: High-dimensional stock portfolio trading with deep reinforcement learning. In: 2022 IEEE Symposium on Computational Intelligence for Financial Engineering and Economics (CIFEr). pp. 1–8. IEEE (2022)
23. QuantConnect (2025), <https://www.quantconnect.com/>
24. Rubesam, A.: Machine learning portfolios with equal risk contributions: Evidence from the brazilian market. *Emerging Markets Review* **51**, 100891 (2022)
25. Shavandi, A., Khedmati, M.: A multi-agent deep reinforcement learning framework for algorithmic trading in financial markets. *Expert Systems with Applications* **208**, 118124 (2022)
26. Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., Riedmiller, M.: Deterministic policy gradient algorithms. In: International conference on machine learning. pp. 387–395. Pmlr (2014)
27. Sutton, R.S., Barto, A.G., et al.: Reinforcement learning: An introduction, vol. 1. MIT press Cambridge (1998)
28. Thrun, S., Schwartz, A.: Issues in using function approximation for reinforcement learning. In: Proceedings of the 1993 connectionist models summer school. pp. 255–263. Psychology Press (2014)
29. Van Hasselt, H., Guez, A., Silver, D.: Deep reinforcement learning with double q-learning. In: Proceedings of the AAAI conference on artificial intelligence. vol. 30 (2016)
30. Wang, H., Zhou, X.Y.: Continuous-time mean–variance portfolio selection: A reinforcement learning framework. *Mathematical Finance* **30**(4), 1273–1308 (2020)
31. Yang, H., Liu, X.Y., Zhong, S., Walid, A.: Deep reinforcement learning for automated stock trading: An ensemble strategy. In: Proceedings of the first ACM international conference on AI in finance. pp. 1–8 (2020)
32. Zhang, Y., Zhao, P., Wu, Q., Li, B., Huang, J., Tan, M.: Cost-sensitive portfolio selection via deep reinforcement learning. *IEEE Transactions on knowledge and data engineering* **34**(1), 236–248 (2020)
33. Zhao, T., Ma, X., Li, X., Zhang, C.: Asset correlation based deep reinforcement learning for the portfolio selection. *Expert Systems with Applications* **221**, 119707 (2023)